TECHNICAL SPECIFICATION | V1.0

FloodSight AI

Disaster Intelligence & Response Platform

DOCUMENT TYPE **Technical Specification**	VERSION **1.0.0**	DATE **June 2026**
STATUS **Production**	PLATFORM **AMD ROCm 7.2**	DEPLOYMENT **Jupyter / Docker**

FloodSight AI is a real-time flood disaster analysis platform combining satellite image processing, multi-agent AI pipelines, GPU-accelerated object detection, and MCP (Model Context Protocol) orchestration to deliver actionable disaster intelligence.

`YOLOv11` `SegFormer` `FastAPI` `Node.js` `React+Vite` `AMD ROCm` `MCP SDK` `OpenCV`

SECTION 1

System Overview

FloodSight AI is an end-to-end disaster intelligence platform that processes satellite and aerial flood imagery through a multi-stage AI pipeline. The system produces real-time assessments of flood severity, population impact, infrastructure damage, and emergency resource requirements within seconds of image submission.

Capability	Technology	Description
Satellite Analysis	YOLOv11 + OpenCV	GPU-accelerated object detection and flood segmentation on aerial imagery
7-Agent Pipeline	MCP SDK + Node.js	Chained agents: Vision - Flood - Geo - Population - Risk - Rescue - Report
Actionable Reports	React + FastAPI	Risk scores, population estimates, rescue plans per image in under 3s
Flood Mapping	SegFormer + HSV	5 HSV water-type signatures optimised for Indian satellite imagery

SECTION 2

Architecture

Three-tier microservices architecture, all services run on a single AMD ROCm GPU server:

```
React Frontend (Vite) :5173
  HTTP Proxy /api -> :8080
Node.js Backend (Express) :8080
  REST routes: /api/analyze/batch, /api/report
  MCP Server (SSE): 7 agents run in sequence
Python AI Microservice (FastAPI) :8090
  YOLOv11  (AMD ROCm CUDA)
  SegFormer (HuggingFace Transformers)
  Satellite Augmentation (OpenCV)
```

MCP Agent Chain

Order	Agent	Input	Output
1	VisionAgent	image file	people, buildings, vehicles, boats, annotated image
2	FloodAgent	image file	flood_pct, roads_damaged, water_depth, flood_mask
3	GeoAgent	EXIF / OSM	district, state, latitude, longitude
4	PopulationAgent	vision + flood	affected_people, displaced, estimation_method
5	RiskAgent	all above	risk_level, risk_score (0-100)
6	RescueAgent	risk + flood	recommendations[], resources_summary, est_hours
7	ReportAgent	all above	key_findings[], object_counts, top_recommendations

SECTION 3

Technology Stack

Layer	Technology	Version	Purpose
Frontend	React + Vite	React 18	Interactive dashboard UI
State Management	Zustand	4.x	Global analysis store
HTTP Client	Axios	1.x	API calls to backend
Backend Runtime	Node.js + Express	18+	REST API and MCP server
MCP Protocol	@modelcontextprotocol/sdk	1.x	SSE-based agent orchestration
AI Service	Python + FastAPI + Uvicorn	3.12 / 0.115	GPU inference endpoints
Object Detection	Ultralytics YOLOv11	yolo11n.pt	COCO-trained 80-class detection
Segmentation	SegFormer (HuggingFace)	nvidia/mit-b2	Flood area segmentation (ADE20K)
Image Processing	OpenCV cv2	4.x	Satellite augmentation pipeline
Deep Learning	PyTorch + ROCm	2.x + 7.2	AMD GPU acceleration
Geocoding	OpenStreetMap Nominatim	REST	Reverse geocoding GPS coordinates
Containerisation	Docker + Docker Compose	latest	Service orchestration

SECTION 4

AI Microservice -- Python FastAPI (:8090)

Endpoints

Method	Path	Input	Output
POST	/detect	Image file + demo flag	Counts, bboxes, annotated image (base64)
POST	/segment	Image file + demo flag	Flood%, roads%, water depth, flood mask
POST	/compare	Before + after images	New flood area, buildings lost, diff image
POST	/exif	Image file	GPS coordinates [lat, lon]
GET	/health	--	GPU status, model info, demo_mode flag

Info: Demo mode activates automatically when torch.cuda.is_available() returns False. Results use OpenCV mock generation with deterministic seeds for reproducibility without GPU hardware.

COCO to FloodSight Class Mapping

COCO ID	COCO Name	FloodSight Category	Satellite Reliability
0	person	people	Fails -- sub-pixel at altitude; occupancy model used instead
2	car	vehicles	Good -- top-down view works well

COCO ID	COCO Name	FloodSight Category	Satellite Reliability
5	bus	vehicles	Good -- large enough to detect
7	truck	vehicles	Good -- large enough to detect
8	boat	boats	Partial -- side-profile trained only; augmentation adds top-down
none	(not in COCO)	buildings	Always uses OpenCV contour detection

Warning: YOLO was trained on ground-level COCO images. At satellite altitude, people are 1-3 pixels and building is not a COCO class. The Satellite Augmentation layer compensates for this.

SECTION 4 -- CONTINUED

4.2 Satellite Image Augmentation

The _satellite_augment_detection() function runs after every YOLO inference, adding satellite-specific detections that YOLO cannot produce. Operates on 1024x1024 resized input.

Building Detection Parameters

Parameter	Value	Rationale
Image resize	1024x1024 px	Normalises scale across satellite zoom levels
Gaussian blur	3x3 kernel	Removes noise before edge detection
Canny thresholds	30 low / 100 high	Wide range for varying building contrast
Dilation	3x3, 1 iteration	Connects broken building outline edges
Area range	150 to 15,000 px sq	Small huts to large commercial buildings
Aspect ratio	0.2 to 5.0	Rejects roads and river banks
Vegetation filter	green > red x1.2 AND green > 60	Removes tree canopy false positives
Water filter	HSV saturation < 30 AND blue > red	Removes open water surfaces
Max results	120 buildings	Hard cap prevents runaway false positives

People Estimation -- Occupancy Model

SimpleBlobDetector was tried but produced 1000+ noise detections (minArea=4px fires on every texture pixel). Replaced with a physically-grounded model:

```
flood_pct_approx = clamp(vehicles x 0.05 + buildings x 0.003 + 0.08, 0.05, 1.0)
people = buildings x 4.5 occupants x flood_pct_approx x 0.03 visible
people += vehicles  # each vehicle has at least a driver
people = min(people, 150)  # hard cap: realistic for a 5 sq km satellite tile
# Example: 42 buildings + 4 vehicles --> (42 x 4.5 x 0.22 x 0.03) + 4 = 19 people
```

Boat Detection -- Calibrated Parameters (Before vs After Fix)

Parameter	Before (Wrong)	After (Fixed)	Reason for Change
Min area	60 px sq	200 px sq	Eliminate noise and debris patches
Min aspect ratio	1.4:1	2.0:1	Square shapes = buildings, not boats
Min dimension	6x6 px	10x10 px	Remove single-pixel noise blobs
Variance threshold	> 80	> 200	Boats have structure; plain water is uniform
Water fraction	25%	45%	Must be clearly ON water, not near a puddle
Hard cap	20 boats	8 boats	Realistic max for a single satellite tile

Annotation Colour Legend

Object	Colour	Shape	Detection Method
Buildings (satellite)	Orange (255,140,0)	Thin rectangle	OpenCV contour analysis
People (estimated)	Red (0,0,255)	Filled dot and ring	Occupancy model at building centroids
Boats (satellite)	Bright green (0,220,60)	Diamond shape	Water contour analysis
Roads	Cyan (0,255,255)	Line segments	Hough transform
YOLO vehicles	Yellow (255,255,0)	Thick rectangle	YOLOv11 neural network
YOLO people	Red (0,0,255)	Thick rectangle	YOLOv11 neural network

SECTION 4 -- CONTINUED

4.3 SegFormer Flood Segmentation

Uses nvidia/mit-b2 SegFormer when HuggingFace Transformers is available on GPU. Falls back to a multi-range OpenCV HSV heuristic optimised for Indian satellite flood imagery.

OpenCV HSV Water Masks (5 ranges)

Water Type	HSV Lower	HSV Upper	Scenario
Clear blue/cyan	[85, 30, 30]	[135, 255, 255]	Clean rivers, sky reflection
Muddy brown/turbid	[8, 30, 30]	[35, 255, 210]	Bihar/Assam silty floodwater
Dark deep water	[95, 20, 10]	[145, 180, 120]	Deep floodwater, navy blue
Murky green-brown	[30, 20, 20]	[75, 180, 160]	Vegetation-mixed floods
Grey-blue (overcast)	channel var < 25, blue >= red	brightness 30-180	Shallow urban inundation

Derived Metrics

Metric	Formula
Damaged roads %	min(flood_pct x 0.85, 95.0)
Damaged buildings %	min(flood_pct x 0.70, 90.0)
Water depth (metres)	max(0.2, flood_pct / 25.0)
Flooded area (sq km)	flood_pct x 0.15 -- assumes 15 sq km tile

SECTION 5

Population Agent -- India Census Density Model

Profile	Density (people/km sq)	Avg Occupancy	Trigger Condition
very_high	2,500	5.2	buildings > 100
high	1,200	4.8	High-density state OR buildings > 50
medium	600	4.5	buildings 20 to 50
low	150	4.2	buildings < 20 (default)

High-density states: West Bengal, Kerala, Uttar Pradesh, Bihar, Tamil Nadu

```
affectedHouseholds = buildings x (flood_pct / 100)
affectedPeople     = affectedHouseholds x profile.occupancy

# Fallback when buildings = 0 (satellite imagery limitation)
# Assumes tile = 5 sq km; 900 people/km sq for high-density states, 450 otherwise
affectedPeople = 5.0 x density x (flood_pct / 100)  [when buildings = 0]

displacedPeople = affectedPeople x max(0, (flood_pct-30)/70)  [when flood > 30%]
```

Risk Agent -- Composite Scoring

Component	Weight	Formula
Flood Coverage	35%	min(flood_pct, 100)
Population Impact	30%	min((affected/500 x 50) + (people_detected/50 x 50), 100)
Road Accessibility	20%	min(damaged_roads_pct x 1.1, 100)
Building Damage	15%	min(damaged_buildings_pct, 100)

Level	Score Range	Response Action
CRITICAL	75 to 100	Mass evacuation, NDRF activation, aerial rescue operations
HIGH	50 to 74	Phased evacuation, rescue boats deployed, medical teams
MEDIUM	25 to 49	Prepare evacuation plans, issue community warnings
LOW	0 to 24	Monitor continuously, maintain alert status

Info: Score boost: if boats > 3, multiply composite score by 1.08 (capped at 100). Active rescue boats indicate an ongoing emergency requiring elevated response.

SECTION 6

Rescue Agent -- Planning Engine

Generates up to 8 prioritised rescue recommendations based on risk level, flood coverage, detected resources, and population estimates. Calculates resource requirements and timeline.

Resource Calculation Formulas

```
rescue_boats    = max(5, min(20, floor(flood_pct / 5)))
helicopters     = 3 if CRITICAL else 1
medical_teams   = 2 if CRITICAL or HIGH else 1
food_packets    = max(500, affected_people)
water_liters    = max(5000, affected_people x 10)
relief_shelters = max(1, floor(displaced_people / 200))
est_hours       = {CRITICAL:72, HIGH:48, MEDIUM:24, LOW:12}[risk_level]
```

Priority	Urgency	Example Actions
IMMEDIATE	< 2 hrs	Deploy rescue boats, activate NDRF, airlift from rooftops
URGENT	2 to 8 hrs	Medical teams, food and water delivery, emergency shelters
MODERATE	8 to 24 hrs	Structural assessment, road clearing, communication units
LOW	> 24 hrs	Aerial survey, community recovery planning

SECTION 7

API Reference

Node.js REST Endpoints (:8080)

Method	Endpoint	Description
POST	/api/analyze/batch	Full batch analysis -- runs all 7 agents per image
POST	/api/analyze/single	Single image analysis
GET	/api/health	Backend and AI service health status
GET	/api/sample-images	List available sample images
GET	/api/results/:batchId	Retrieve cached batch analysis result
GET	/mcp	MCP SSE connection endpoint
POST	/mcp?sessionId=	MCP tool invocation (JSON body)

MCP Tool Registry

Tool Name	Key Inputs	Description
detect_objects	image_path	YOLO + satellite augmentation pipeline
analyze_flood	image_path	SegFormer or OpenCV flood segmentation
get_location	image_path, lat?, lon?	EXIF GPS extraction + Nominatim reverse geocode

Tool Name	Key Inputs	Description
estimate_population	buildings, flood_pct, state?	India census density population model
assess_risk	flood_pct, roads, people, buildings	Composite 0-100 risk score
generate_rescue_plan	risk_level, affected_people	Prioritised rescue recommendations
compile_report_data	batch_id	Compile full analysis report from all agents
compare_flood_images	before_path, after_path	Before and after change detection
run_full_pipeline	image_path, filename?	Execute all 7 agents in sequence

SECTION 8

Configuration and Environment Variables

Variable	Service	Default	Description
PORT	Node.js	8080	Backend HTTP port
AI_SERVICE_URL	Node.js	http://localhost:8090	Python AI service base URL
VLLM_HOST	Node.js	http://localhost:8000	vLLM / Qwen3 endpoint
LLM_MODEL	Node.js	Qwen/Qwen3-8B-Instruct	Language model for narratives
DATA_DIR	Node.js	./data	Root path for uploads, results, reports
AI_SERVICE_PORT	Python	8090	FastAPI server port
DEVICE	Python	auto	auto | cuda | cpu
DEMO_MODE	Python	auto	auto | true | false
YOLO_MODEL_PATH	Python	yolo11n.pt	Path to YOLO weights file
SEGFORMER_MODEL	Python	nvidia/mit-b2	HuggingFace model identifier

SECTION 9

Deployment Guide

Jupyter Notebook Environment (AMD ROCm)

```
pip install ultralytics transformers torch torchvision opencv-python-headless fastapi uvicorn pillow nu
mpy

# Terminal 1: AI microservice
cd /workspace/shared/Disaster/disator1/ai_service && python main.py
# -> Uvicorn running on http://0.0.0.0:8090

# Terminal 2: Node.js backend
cd /workspace/shared/Disaster/disator1 && npm install && node backend/src/index.js
# -> Express running on http://0.0.0.0:8080

# Terminal 3: React frontend
cd frontend && npm install && npm run dev
# -> Vite running on http://0.0.0.0:5173
```

Vite Config for Jupyter Server Proxy

```
// vite.config.js
export default defineConfig({
  base: '/jupyter-hack-team-XXXX/proxy/5173/',
  server: {
    host: '0.0.0.0', port: 5173,
    proxy: {
      '/api':          { target: 'http://127.0.0.1:8080', changeOrigin: true },
      '/sample_images':{ target: 'http://127.0.0.1:8080', changeOrigin: true },
      '/health':       { target: 'http://127.0.0.1:8080', changeOrigin: true }
    }
  }
```

```
})
```

SECTION 10

Known Limitations and Roadmap

Current Limitations

Limitation	Impact	Workaround
YOLO trained on ground-level COCO	Cannot detect buildings from satellite	OpenCV satellite augmentation layer
People are sub-pixel at altitude	YOLO people count always 0	Building-occupancy model (+-50% accuracy)
SegFormer ADE20K class 21 only	May miss non-standard water colors	5-range OpenCV HSV fallback
Boat top-view not in YOLO training	Low boat detection from YOLO alone	Water-contour analysis with strict filters
No temporal flood tracking	Cannot show flood progression over time	Before/After compare endpoint available
Geo uses filename heuristic fallback	Inaccurate without EXIF GPS data	Embed GPS in EXIF before uploading image
Population count is estimate only	+-50% accuracy	Clearly labelled as Estimated in the UI

Roadmap

Feature	Description	Priority
Fine-tuned satellite model	Train YOLOv11 on DOTA/xView/SpaceNet aerial datasets	HIGH
Sentinel-2 SAR integration	All-weather cloud-penetrating flood mapping via SAR	HIGH
Real-time satellite feed	ISRO RISAT / Resourcesat-2A API for live tile ingestion	MEDIUM
LLM narrative reports	Qwen3-8B generates situation briefings via vLLM	MEDIUM
Multi-date time series	Flood progression tracking over 24h to 7-day window	LOW
Mobile app	iOS/Android field responder app for rescue teams	LOW